CYBERSHIELD AI: A WEB-BASED REAL-TIME CYBERSECURITY MONITORING DASHBOARD WITH AI-ASSISTED THREAT ANALYSIS

Project Phase I Report

submitted to the APJ Abdul Kalam Technological University

in partial fulfillment of the requirements for the award of the degree

Bachelor of Technology

in

Computer Science and Engineering

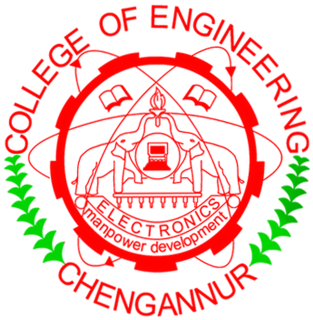
by

SREEDEVI R P (CHN23CS181)

VISHNU V NAIR (CHN23CS196)

ABHI RAJ (LCHN23CS197)

Project Group 16



Department of Computer Engineering

College of Engineering Chengannur

OCTOBER 2026

DEPARTMENT OF COMPUTER ENGINEERING

COLLEGE OF ENGINEERING CHENGANNUR

2026-27

CERTIFICATE

This is to certify that the project report titled ***CYBERSHIELD AI: A WEB-BASED REAL-TIME CYBERSECURITY MONITORING DASHBOARD WITH AI-ASSISTED THREAT ANALYSIS*** *is a bonafide record of the* ***CSD 415 PROJECT PHASE I*** *presented by* ***SREEDEVI R P (CHN23CS181), VISHNU V NAIR (CHN23CS196) & ABHI RAJ (1CHN23CS197)****, Seventh Semester B.Tech. students in Computer Science and Engineering, under our guidance and supervision. This report is submitted in partial fulfillment of the requirements for the award of the degree* ***B.Tech. in Computer Science and Engineering*** *of* ***APJ Abdul Kalam Technological University****.*

Smt. Varsha C mohan
(Project Guide)
Assistant Professor
Dept. of Computer Engineering
College of Engineering Chengannur

Dr. Geetha S
(Project Coordinator)
Associate Professor
Dept. of Computer Engineering
College of Engineering Chengannur

Prof. (Dr.) Renu George
Head of the Department
Dept. of Computer Engineering
College of Engineering Chengannur

DECLARATION

We undersigned hereby declare that the project report **"CyberShield AI: A Web-Based Real-Time Cybersecurity Monitoring Dashboard with AI-Assisted Threat Analysis"**, submitted for partial fulfillment of the requirements for the award of degree of Bachelor of Technology of the APJ Abdul Kalam Technological University, Kerala is a bonafide work done by us under the supervision of **Smt. Varsha C Mohan**, Assistant Professor, Department of Computer Engineering. This submission represents our ideas in our own words, and where ideas or words of others have been included, we have adequately and accurately cited and referenced the original sources. We also declare that we have adhered to the ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in our submission. We understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not been previously formed the basis for the award of any degree, diploma, or similar title of any other University.

Sreedevi R P(CHN23CS181)

Place: Chengannur

Vishnu V Nair (CHN23CS196)

Date:14/10/2026

Abhi Raj(LCHN23CS197)

ACKNOWLEDGEMENT

This work would not have been possible without the support and guidance of many people. First and foremost, we thank Almighty God for providing us with the strength, resources, and ability to successfully carry out this project.

We would like to express our sincere gratitude to **Prof. Dr. Hari V S**, The Principal of College of Engineering Chengannur, for providing excellent facilities and creating a conducive environment for the successful completion and presentation of this project. Special thanks also go to HOD **Dr. Renu George**, Professor, Department of Computer Engineering, for the exceptional support, guidance, and encouragement throughout the project.

We are deeply thankful to our project guide **Smt. Varsha C Mohan**, Assistant Professor, Department of Computer Engineering, for her valuable advice, constant guidance, and support throughout the project, and to our project coordinators **Dr. Geetha S**, Associate Professor, **Smt. Suvarna Dev**, Assistant Professor, and **Smt. Neethu A**, Assistant Professor, Department of Computer Engineering, for their timely suggestions.

Finally, we would like to thank our parents, friends and faculty members for their cooperation, encouragement, and moral support, without which this project would not have been completed successfully. Thank you all for your understanding and support.

Sreedevi R P

Vishnu V Nair

Abhi Raj

ABSTRACT

Security Operations Centers (SOCs) and small IT teams face a widening gap between the offline accuracy of standalone machine learning models and the tooling needed to interpret, triage and respond to threats in real time. Academic detectors often reach high accuracy on static datasets, but in live operation they behave as black boxes, generate alert fatigue and delay notification. This project presents **CyberShield AI**, a unified, web-based, real-time cybersecurity monitoring dashboard with integrated Explainable AI (XAI) threat analysis. The platform combines four capabilities. First, a **two-way dual-engine phishing detector** queries a cloud-hosted model trained on 235,000 verified URLs and, in parallel, a local 15-feature lexical analyzer (Shannon entropy, subdomain depth, credential-harvesting keywords), and reconciles the two through a consensus decision matrix. Second, **zero-knowledge malware forensics** computes SHA-256 hashes inside the browser using the Web Crypto API and correlates them with 72 antivirus vendor engines, so that no file is uploaded. Third, **real-time network telemetry** provides simulated 5-tuple flow events, live bandwidth charts and anomaly scores. Fourth, **explainable alerting** uses Google Gemini to turn numerical indicators into two-sentence analyst rationales and dispatches critical incidents to Discord and Slack webhooks. The front end is built with React, Vite and Tailwind CSS, the back end with Node.js, Express and TypeScript, and persistence and role-based access control with Google Cloud Firestore. Preliminary evaluation reports a dual-engine consensus accuracy of 98.2%, a false-positive rate below 1.8% and an end-to-end scan turnaround below 2.0 seconds.

VISION AND MISSION OF THE INSTITUTION

VISION

To emerge as an institution of excellence that generates knowledge through teaching, learning and research for the inclusive development of the society.

MISSION

M1. To impart engineering education that enables the students to design, develop and maintain systems, products and processes for the society, utilising the evolving technology.

M2. To orient students for professional success, critical thinking and equity.

M3. To inculcate the students with entrepreneurial skills, ethical and social values.

VISION AND MISSION OF THE DEPARTMENT

VISION

Emerge as a meritorious centre in computing to mould competent and socially committed professionals.

MISSION

M1: Nurture a stimulating environment by means of quality education and state of the art facilities to groom young minds for their professional career, higher education and innovative research.

M2: Foster managerial and entrepreneurial skills of students through co-curricular and extra-curricular activities.

M3: Facilitate the students to address socially relevant problems with professional ethics and values.

PROGRAMME EDUCATIONAL OBJECTIVES (PEOs)

PEO1: Graduates shall have a sound knowledge in Computer Science adaptable to the evolving technical challenges to pursue a profession in computing.

PEO2: Graduates shall be theoretically proficient for higher studies and contribute to the advancements in computing through research.

PEO3: Graduates shall have leadership and teamwork qualities competent of being good entrepreneurs.

PEO4: Graduates shall exhibit social commitment with ethical values in designing computing solutions.

PROGRAMME OUTCOMES (POs)

1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

2. **Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

3. **Design/Development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

4. **Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

5. **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.

6. **The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

7. **Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

9. **Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

10. **Communication:** Communicate effectively on complex engineering activities with the

engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

11. **Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

12. **Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAMME SPECIFIC OUTCOMES (PSOs)

PSO1: Able to analyze real world situations to develop software solutions incorporating new ideas and modern technology.

PSO2: Competent to Design and develop digital hardware based programming solutions.

CONTENTS

Declaration ii

Acknowledgement iii

Abstract iv

Vision and Mission of the Institution v

Vision and Mission of the Department vi

Programme Educational Objectives (PEOs) vii

Programme Outcomes (POs) viii

Programme Specific Outcomes (PSOs) x

List of Figures xiv

List of Tables xv

List of Abbreviations xvi

1 INTRODUCTION 1

1.1 Background and Motivation 1

1.2 Problem Statement 1

1.3 Objectives of the Project 2

1.4 Scope of the Project 2

1.5 Organization of the Report 3

2 LITERATURE REVIEW 4

2.1 Review of Existing Research Works 4

2.2 Comparative Analysis of Existing Methods 5

2.3 Limitations of Existing Systems 6

2.4 Research Gap and Motivation 6

3 REQUIREMENTS ANALYSIS **7**

3.1 Problem Analysis 7

3.2 Existing System 7

3.3 Proposed System 7

3.4 Functional Requirements 8

3.5 Non-functional Requirements 9

3.6 Hardware and Software Requirements 9

3.6.1 Hardware Requirements 9

3.6.2 Software Requirements 10

4 PROPOSED METHODOLOGY AND SYSTEM DESIGN **11**

4.1 Proposed Methodology 11

4.2 System Architecture / Block Diagram 11

4.3 Module Description 13

4.4 Algorithm / Model Design 14

4.4.1 Shannon Entropy 14

4.4.2 15-Feature Lexical Extractor 14

4.4.3 Random Forest Classifier 15

4.4.4 Dual-Engine Consensus Decision Matrix 15

4.4.5 Overall Scan Algorithm 15

4.5 Data Flow Diagram / UML Diagrams 16

4.5.1 Use Case Diagram 16

4.5.2 Level-0 Data Flow Diagram 16

4.5.3 Level-1 Data Flow Diagram 17

4.6 Database Design 18

5 PRELIMINARY ANALYSIS AND DESIGN **19**

5.1 Design Overview 19

5.2 Tools and Technologies 19

5.3 Feasibility Study 20

5.3.1 Technical Feasibility 20

5.3.2 Economic Feasibility 20

5.3.3 Operational Feasibility 20

5.3.4 Legal and Ethical Feasibility 20

5.4 Preliminary Analysis / Prototype 20
5.5 Preliminary Results 21
5.5.1 Quantitative Results 21
5.5.2 User Interface Screenshots 21

6 PROJECT PLANNING AND NBA MAPPING 26
6.1 Work Breakdown Structure 26
6.2 Project Timeline / Gantt Chart 26
6.3 Task Allocation among Team Members 28
6.4 Expected Project Outcomes 31
6.4.1 Course Outcomes 31
6.4.2 NBA PO and PSO Mapping 31

7 CONCLUSION AND FUTURE WORK 33
7.1 Summary of Phase I Work 33
7.2 Limitations 33
7.3 Future Work / Plan for Phase II 34
7.3.1 Future Scope 34
7.3.2 Work Planned for Phase II 34

REFERENCES 36

Appendices 38

LIST OF FIGURES

4.1 System architecture of CyberShield AI 12
4.2 Two-way dual-engine consensus processing pipeline 13
4.3 Zero-knowledge client-side hashing workflow 13
4.4 Use case diagram of CyberShield AI 16
4.5 Data flow diagram – Level 0 17
4.6 Data flow diagram – Level 1 17
4.7 Cloud Firestore collection relationships 18

5.1 Authentication dialog (email/password and Google sign-in) 21
5.2 SOC executive command dashboard 22
5.3 Two-way phishing detection interface with Engine 1 and Engine 2 side by side 22
5.4 Malware hash analysis with client-side SHA-256 and 72-engine result 23
5.5 Network telemetry with live bandwidth and protocol distribution 23
5.6 Threat-intelligence feeds with searchable CVEs and indicators 24
5.7 SOC alerting and webhook integration settings 24
5.8 Settings, identity and role status 25

6.1 Work breakdown structure tree 26
6.2 CyberShield AI project timeline and Gantt chart (Phase I: 24 July 2026 – 9 October 2026; Phase II: planned, November 2026 – April 2027) 27

LIST OF TABLES

2.1 Gap analysis of the 12 reviewed papers 5

3.1 Problem aspects and the corresponding system response 7
3.2 Functional requirements specification 8
3.3 Hardware and cloud deployment specifications 9
3.4 Software development stack 10

4.1 Lexical and structural URL features 14
4.2 Dual-engine cross-verification consensus decision matrix 15
4.3 Firestore Collection Structure 18

5.1 Tools and technologies 19
5.2 Quantitative performance and latency results 21

6.1 Work breakdown structure 26
6.2 Team task allocation matrix for Phase I 28
6.3 Periodic task allocation and progress log from the project diary (Group 16) . 28
6.4 Course outcome to programme outcome (CO-PO-PSO) mapping 31

ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
AV	Antivirus
C2	Command and Control
CISA	Cybersecurity and Infrastructure Security Agency
CNN	Convolutional Neural Network
CVE	Common Vulnerabilities and Exposures
DFD	Data Flow Diagram
DGA	Domain Generation Algorithm
eBPF	Extended Berkeley Packet Filter
FAIR	Factor Analysis of Information Risk
GNN	Graph Neural Network
HTTPS	Hypertext Transfer Protocol Secure
IoC	Indicator of Compromise
JSON	JavaScript Object Notation
KEV	Known Exploited Vulnerabilities
LLM	Large Language Model
ML	Machine Learning
NBA	National Board of Accreditation
OSINT	Open-Source Intelligence
OTX	Open Threat Exchange
PE	Portable Executable
RBAC	Role-Based Access Control
REST	Representational State Transfer
RF	Random Forest
ROSI	Return on Security Investment
SHA	Secure Hash Algorithm
SIEM	Security Information and Event Management
SOAR	Security Orchestration, Automation and Response
SOC	Security Operations Center
SPA	Single-Page Application
SRS	Software Requirements Specification
TLS	Transport Layer Security
URL	Uniform Resource Locator
WBS	Work Breakdown Structure
XAI	Explainable Artificial Intelligence

CHAPTER 1

INTRODUCTION

1.1 Background and Motivation

Modern organizations operate in an adversarial landscape where digital perimeters are continuously probed by automated exploits, targeted spear-phishing campaigns and evasive malware. Phishing and social engineering account for over 85% of initial-access footholds in enterprise breaches. At the same time, malware authors use in-memory code injection, living-off-the-land binaries and packers or crypters that bypass legacy signature scanning [7, 8].

Researchers have developed machine learning, deep learning and large language model (LLM) detectors that exceed 95% accuracy in offline experiments [1, 2, 3, 5, 6]. There is, however, a **research-to-operations disconnect**:

1. Research systems typically run as command-line scripts or offline notebooks.
2. They return binary labels or raw probabilities without human-interpretable reasoning.
3. Junior analysts and small IT teams are overwhelmed by fragmented consoles: one tool for URLs, another for file hashes, a third for network traffic and another for threat feeds.
4. Commercial SIEM and SOAR suites are costly, resource-heavy and need dedicated engineers [10, 11].

These limitations show the need for a unified, lightweight, explainable and web-accessible SOC platform for agile IT teams.

1.2 Problem Statement

To address fragmented cybersecurity tools and unexplained threat alerts by integrating dual-engine phishing detection, malware analysis, AI-based threat explanations and real-time security alerting into a unified monitoring platform. Four deficiencies of existing solutions motivate this work:

- **Black-box predictions:** analysts cannot tell whether a link was flagged for typosquatting, domain randomness or deceptive keywords.

- **Tool fragmentation:** siloed telemetry increases the time to detect and to respond.
- **Privacy risk in file analysis:** uploading suspicious documents to public scanners can expose confidential data.
- **Delayed escalation:** high-risk findings outside business hours remain in local logs without an automated bridge to incident channels.

1.3 Objectives of the Project

The goal of **CyberShield AI** is to bridge machine-learning detection and human-centred incident triage in one full-stack platform. The objectives of Phase I are:

1. **Dual-engine phishing scanner:** combine a cloud model trained on 235,000 verified URLs with a fast local 15-feature lexical analyzer to cross-verify threats and lower false positives.
2. **Explainable AI:** use Google Gemini to produce two-sentence natural-language threat rationales in SOC terminology.
3. **Zero-knowledge malware hashing:** compute SHA-256 in the browser through the Web Crypto API and query 72 antivirus engines without uploading file contents.
4. **Live network telemetry:** simulate kernel eBPF probes over 5-tuple flow records to compute anomaly scores and visualize bandwidth.
5. **Threat-feed aggregation:** normalize IoCs from CISA KEV, AlienVault OTX, CertStream and VirusTotal into an instantly searchable feed.
6. **Automated escalation:** send colour-coded notifications to Discord and Slack within 2 seconds of a high-threat detection.
7. **Role-based access control:** provide persistent storage in Google Cloud Firestore and separate administrator governance from analyst workspaces.

1.4 Scope of the Project

In scope (Phase I):

- Full-stack web application using React, Vite, Node.js, Express and TypeScript.
- Dual-engine phishing pipeline with consensus scoring.

- SHA-256 file hashing and multi-engine malware reputation queries.
- Real-time network telemetry visualizer with ping and download speed tests.
- Live threat-intelligence aggregator with instant client-side filtering.
- Outbound Discord/Slack webhook alerts for high-severity events.
- Authentication and user-profile synchronization in Firebase Firestore.

Out of scope (deferred to Phase II):

- A dedicated endpoint agent running on user operating systems.
- Dynamic sandbox execution for live malware detonation.
- A Manifest V3 Chromium/Firefox extension for URL interception.

1.5 Organization of the Report

Chapter 1 presents the background, problem statement, objectives and scope. Chapter 2 reviews 12 research papers, compares them and identifies the research gap. Chapter 3 covers problem analysis, the existing and proposed systems, functional and non-functional requirements, and hardware and software requirements. Chapter 4 describes the methodology, architecture, modules, algorithms, data flow and use case diagrams, and the Firestore schema. Chapter 5 presents the preliminary design, tools, feasibility study, prototype and results. Chapter 6 covers the work breakdown structure, timeline, task allocation, expected outcomes and NBA mapping. Chapter 7 concludes Phase I, lists limitations and outlines Phase II. References and appendices follow.

CHAPTER 2

LITERATURE REVIEW

2.1 Review of Existing Research Works

The design of CyberShield AI is based on a review of 12 recent publications on phishing detection, malware analysis, explainable AI and human-centred security. Figures quoted below are those reported by the authors on their own datasets and are not directly comparable.

Misiek and Hyla extracted lexical URL features and trained an XGBoost classifier, evaluating generalization across several datasets [1]. They report 90.7% accuracy and an F1-score of 91.2%, but the work is limited to offline evaluation and has no real-time alert notification.

Koide et al. proposed *ChatSpamDetector*, which decodes and simplifies emails, analyzes them with an LLM and returns a verdict with an explanation [2]. It reports 99.70% accuracy. LLM inference adds latency, so it is difficult to use alone for real-time filtering without fast heuristic guardrails, and the explanations are not tailored to analyst triage.

Aldhuwayhi and Alromih combined machine-learning URL classification with crowdsourced threat intelligence and report 99.8% accuracy [3]. Static classifiers adapt poorly to emerging phishing campaigns, which motivates a human-in-the-loop verification loop.

Duarte et al. studied early detection of phishing URLs in parked and newly registered domains [4], reporting 97.28% accuracy and 96.02% recall under 10-fold cross-validation. Newly generated domains can evade blacklists, so lexical features and entropy remain useful.

Elgarduh et al. proposed Hybrid Feature Image Mapping (HFIM), which encodes structural, semantic and behavioural malware features as RGB images classified by CNNs, with 99.0% accuracy [5]. The computational cost makes it impractical for browser-based, rapid analyst screening.

Gutierrez et al. applied deep learning to malware features for real-time detection and report precision of 98%, recall of 97% and an F1-score of 97.5% [6]. High latency and interface bottlenecks remain.

Kara and Yılmaz presented *HAP-Analyzer*, a hybrid static, dynamic and memory analysis

approach for fileless malware [7]. Such malware resides in process memory, so kernel-level telemetry is needed, which makes the tooling complex.

Alshoulie and Mehmood surveyed deep-learning malware detection and found detection rates of about 93% to 98.7% across the surveyed work, with adversarial perturbation and evasion still largely unaddressed [8].

Almarwani et al. studied human factors and found that awareness of risk does not lead to protective behaviour unless users have actionable detection competence and understandable feedback [9].

Khattak et al. proposed a strategic model for cyber-security challenges in software development organizations, identifying unpatched external gateways and credential theft as major risk vectors for small and medium teams [10].

He et al. applied the FAIR model to Return on Security Investment (ROSI), showing that automated threat interception can reduce breach-remediation costs [11].

Abril et al. explored simulation-based approaches to cybersecurity, reporting that interactive simulated threat environments improve retention of security practice compared with passive material [12].

2.2 Comparative Analysis of Existing Methods

Table 2.1 compares the reviewed studies and shows how CyberShield AI responds to each gap.

Table 2.1: Gap analysis of the 12 reviewed papers

Ref.	Approach	Reported result	Research gap	CyberShield AI response
[1]	XGBoost on lexical URL features; cross-dataset validation	Accuracy 90.7%, F1 91.2%	No real-time alert notification	Real-time Discord and Slack webhook alerts
[2]	LLM-based email analysis with explanation	Accuracy 99.70%	LLM latency; limited analyst-oriented explanation	Heuristics in under 12 ms; Gemini gives plain-English rationale
[3]	ML URL classifier with crowdsourced threat intelligence	Accuracy 99.8%	Difficulty adapting to emerging threats	Crowdsourced verification loop
[4]	ML on parked and newly registered domain features	Accuracy 97.28%, recall 96.02%	New domains evade blacklists	15 URL features and Shannon entropy
[5]	Malware features as RGB images with CNN	Accuracy 99.0%	High computational cost limits browser use	Client-side SHA-256 hashing and 72+ AV engines
[6]	Deep learning on malware features	Precision 98%, recall 97%, F1 97.5%	High latency and interface bottlenecks	Fast lexical analysis with asynchronous cloud AI
[7]	Hybrid static, dynamic and memory analysis	Not quoted	Needs kernel-level tooling; complex interface	Simulated eBPF telemetry on a central dashboard
[8]	Survey of deep-learning malware detection	About 93–98.7% (surveyed range)	Opacity; adversarial vulnerability	Two-way cross-verification consensus matrix
[9]	Behavioural study of detection competence	Not applicable	Ignores analyst tooling usability	Human-centred responsive SOC interface
[10]	Strategic cyber-risk model for software organizations	Not applicable	Fragmented, unaffordable commercial tools	All-in-one lightweight command center

Ref.	Approach	Reported result	Research gap	CyberShield AI response
[11]	FAIR-based ROSI economics	Not applicable	Disconnected from live technical telemetry	Dashboard tracks prevented threats
[12]	Simulated threat environments	Not applicable	Passive training does not build competence	Actionable technical explanations for every threat

2.3 Limitations of Existing Systems

- **Isolated detection pipelines:** URL scanners do not correlate with network flows, and malware detectors do not push webhooks.
- **No explainability:** a high score does not tell the analyst which tokens or domain properties triggered it.
- **High resource overhead:** commercial SOC suites need large memory, high-bandwidth ingestion and dedicated administration.
- **Data exposure:** conventional cloud scanners require uploading the whole executable, with confidentiality and regulatory risk.
- **Offline focus:** most studies stop at offline evaluation without real-time alerting.

2.4 Research Gap and Motivation

Research has advanced raw predictive accuracy, but none of the reviewed work integrates lightweight dual-engine phishing detection, zero-knowledge browser hashing, real-time flow telemetry and generative AI explanations in one accessible command center. CyberShield AI is built to bridge this operational gap.

CHAPTER 3

REQUIREMENTS ANALYSIS

3.1 Problem Analysis

A SOC workflow needs four sequential operations: **detect** suspicious artifacts (URLs, binaries, abnormal connections), **analyze** them into verifiable indicators, **explain** numerical scores as actionable intelligence, and **respond** by alerting the team. Legacy workflows fail because these steps run on disjointed platforms. Table 3.1 maps each problem aspect to the response of CyberShield AI.

Problem aspect	**Response in CyberShield AI**
Single-model detectors give unexplained, error-prone verdicts	Dual-engine consensus with Gemini rationale
Tools are fragmented across URLs, files, network and feeds	One dashboard with all modules
Uploading files to scanners leaks data	Browser-side SHA-256; only the hash is sent
High-risk findings are not escalated promptly	Automatic Discord and Slack webhooks
Commercial suites are costly and heavy	Lightweight serverless-tier stack
Third-party APIs may fail	Fallback to local heuristic scoring

Table 3.1: Problem aspects and the corresponding system response

3.2 Existing System

In the existing ecosystem, analysts manually copy suspicious links into command-line scripts or public blocklist checkers. Binary inspection requires uploading files to third-party portals, which takes minutes and risks leaking proprietary data. Network monitoring runs in complex terminals such as Wireshark and tcpdump, separate from user-facing dashboards. Communication is manual: analysts copy scan results, draft messages and send them to the team by email or chat.

3.3 Proposed System

CyberShield AI introduces:

- a **two-way dual-engine phishing engine** that evaluates a URL against a 235k-sample cloud model and a local 15-feature lexical analyzer in parallel and applies a consensus matrix to produce a combined risk score;
- a **client-side SHA-256 pipeline** that fingerprints files locally and queries 72 antivirus engines without uploading the file;
- a **network telemetry stream** showing simulated eBPF 5-tuple flow events and live bandwidth;
- an **explainable AI layer** using Google Gemini for natural-language reasoning;
- **webhook integrations** that broadcast critical events to Discord and Slack in under 2 seconds;
- **Firebase Firestore synchronization** for secure, role-based persistence.

3.4 Functional Requirements

ID	Feature	Specification
1	Dual-engine phishing scan	Accept a URL, validate its syntax and run Engine 1 (PhishGuard 235k cloud model) and Engine 2 (CyberShield 15-feature lexical analyzer) simultaneously.
2	Consensus decision matrix	Compute a weighted combined score and evaluate the agreement state (`FULL_AGREEMENT` or `DISCORDANT`), classifying the result as `CONFIRMED_PHISHING`, `SUSPICIOUS` or `VERIFIED_SAFE`.
3	Explainable AI rationale	Call the Google Gemini API for a two-sentence threat summary within a 2.5-second timeout, falling back gracefully if unavailable.
4	Zero-knowledge malware forensics	Compute SHA-256 in the browser with `crypto.subtle.digest()` and query the back end to correlate with 72 antivirus engines.
5	Real-time network telemetry	Stream simulated 5-tuple events, refresh bandwidth charts every 5 seconds and compute anomaly scores from 0 to 100.
6	Threat intelligence aggregator	Normalize and index indicators from CISA KEV, AlienVault OTX, CertStream and VirusTotal with instant in-memory filtering.
7	Automated SOAR alerting	Dispatch colour-coded webhook payloads to Discord and Slack when a scan is rated Critical or High.

Table 3.2: Functional requirements specification

3.5 Non-functional Requirements

1. **Performance:** the local heuristic engine shall run in under 12 ms; an end-to-end phishing scan, including cloud and LLM generation, shall take no more than 2.0 s.

2. **Security and privacy:** user files shall never be transmitted; only client-computed SHA-256 hashes are sent. Administrator routes are enforced by Firestore security rules.

3. **Availability:** if the cloud model or Gemini times out, the platform shall fall back to local heuristic pre-scoring without crashing or blocking the interface.

4. **Usability:** the interface shall offer a dark/light SOC theme, responsive layouts for mobile, tablet and desktop, and colour-coded severity badges.

5. **Maintainability:** the code base shall use TypeScript typing, modular service boundaries and separation of concerns.

3.6 Hardware and Software Requirements

3.6.1 Hardware Requirements

Item	Specification
Development machine	Multicore x86_64 or ARM64 processor (Intel Core i5/i7, AMD Ryzen 5/7 or Apple Silicon), 8 GB RAM minimum (16 GB recommended), 10 GB free SSD space
Cloud hosting	1 vCPU, 1 GB RAM, Linux container runtime (Docker, Google Cloud Run or Render)
Client device	Desktop, laptop or mobile workstation with a modern web browser
Network	10 Mbps or faster connection for external API calls

Table 3.3: Hardware and cloud deployment specifications

3.6.2 Software Requirements

Component	Technology
Operating system	Ubuntu 22.04 LTS (recommended), macOS, Windows 10/11 (WSL2)
Runtime	Node.js 20 LTS or higher; Python 3.10+ for model training scripts
Front end	React 19, TypeScript, Vite 6, Tailwind CSS, Recharts, Lucide React
Back end	Express.js, TypeScript (tsx), Axios
ML / AI	Scikit-learn, Pandas, NumPy, Joblib, tldextract, Google Gemini SDK (`@google/genai`)
Cloud database and auth	Google Firebase Cloud Firestore, Firebase Authentication
External APIs	Google Gemini API, VirusTotal v3 REST API
Protocols	HTTPS, WebSockets (CertStream), REST/JSON, Discord and Slack incoming webhooks

Table 3.4: Software development stack

CHAPTER 4

PROPOSED METHODOLOGY AND SYSTEM DESIGN

4.1 Proposed Methodology

CyberShield AI follows five interconnected engineering methods.

1. **Parallelized dual-engine inference:** the back end sends an asynchronous request to the remote 235k PhishGuard model while computing the 15 local lexical features in memory, using `Promise.allSettled`.

2. **Consensus synthesis:** a weighted decision matrix balances the generalization of the machine-learning model with hard-coded syntactic indicators, preventing edge-case bypasses.

3. **Zero-knowledge in-browser cryptography:** hashing is done on the client with `window.crypto.sub` so no bytes of a file leave the workstation.

4. **Asynchronous telemetry simulation:** network flow tuples are generated and buffered using event-loop timers, emulating kernel ring buffers.

5. **Generative contextualization:** numeric vectors and feature penalties are bundled into a prompt for Gemini, which returns a readable justification.

4.2 System Architecture / Block Diagram

Figure 4.1 shows the architecture. Users (analysts) and the SOC admin use a React and Vite web dashboard, which calls a REST API gateway built with Node.js and Express. The gateway fronts four services: Auth and RBAC, AI Analysis Engine, Threat Intelligence, and Alerts and Webhook. These use ML models (PhishGuard and Random Forest), the Gemini API and VirusTotal, and persist data in Firestore (with an in-memory cache), an audit log store and Discord/Slack webhooks.

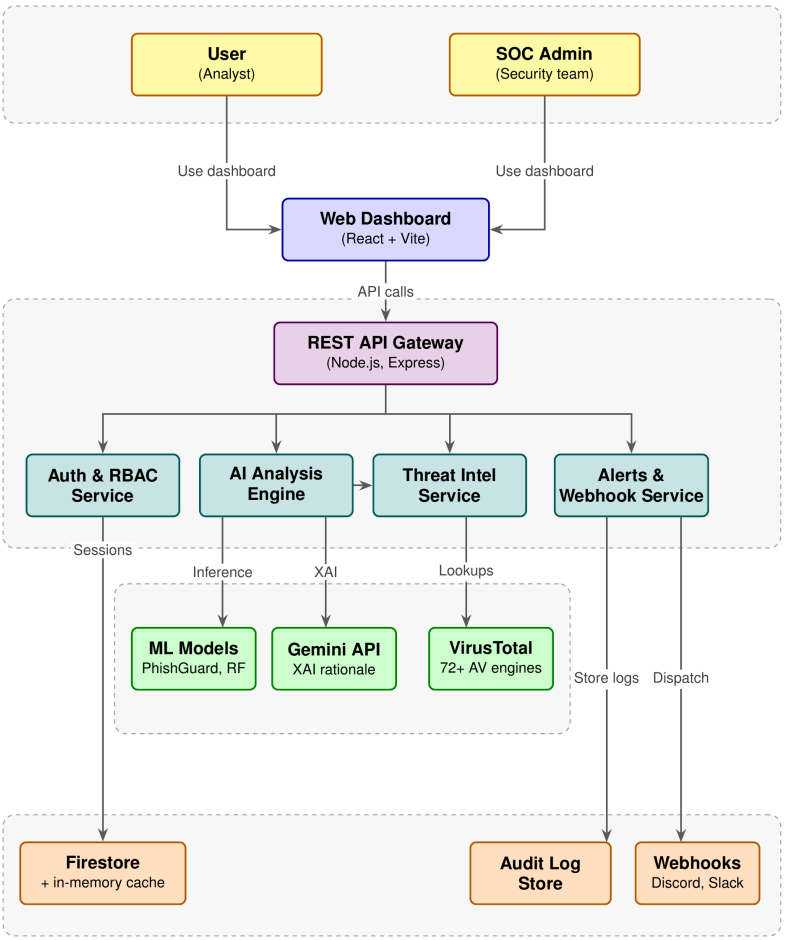


Figure 4.1: System architecture of CyberShield AI

The system is organized in four tiers:

- **Tier 1, presentation:** React SPA with Tailwind CSS, Lucide icons and Recharts, plus the Web Crypto API for SHA-256 hashing.
- **Tier 2, API and gateway:** Express.js with request sanitization, the dual-engine dispatcher, the threat-feed aggregation service and the webhook dispatcher.
- **Tier 3, external services:** Google Gemini (XAI), VirusTotal v3, CISA KEV and AlienVault OTX feeds and the CertStream WebSocket.
- **Tier 4, consensus and persistence:** the consensus decision matrix and Google Cloud Firestore.

Figure 4.2 shows the dual-engine processing pipeline.

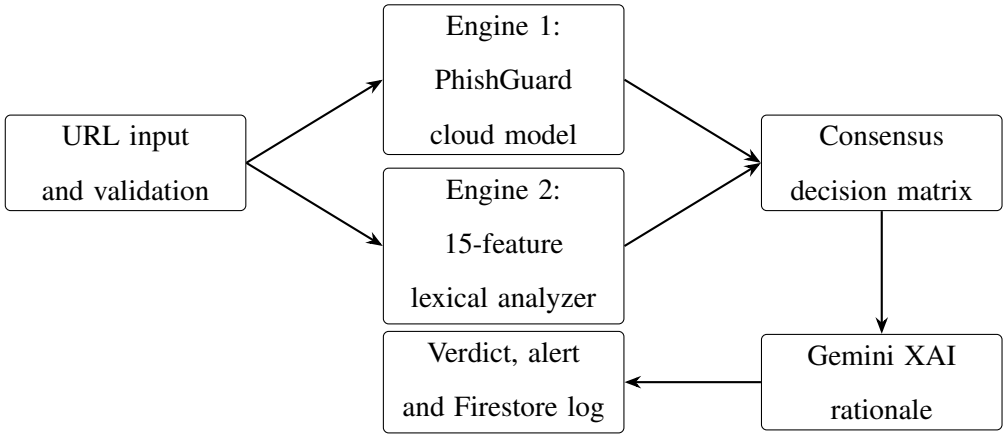


Figure 4.2: Two-way dual-engine consensus processing pipeline

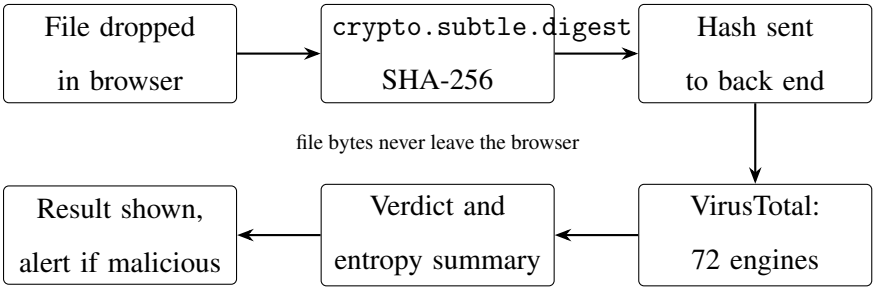


Figure 4.3: Zero-knowledge client-side hashing workflow

4.3 Module Description

The system has nine modules.

- **M1 Dual-engine phishing detection:** validates the URL, calls PhishGuard and the lexical engine in parallel, runs cross-verification and queries Gemini.
- **M2 Zero-knowledge malware forensics:** handles drag-and-drop files, computes browser SHA-256, correlates with 72 antivirus engines and shows signature classifications.
- **M3 Network telemetry and flow anomaly engine:** monitors simulated 5-tuple flows, updates bandwidth charts and computes anomaly scores.
- **M4 Infrastructure vulnerability scanner:** runs simulated port audits and exposure ratings across cloud assets.
- **M5 Threat-intelligence (OSINT) aggregator:** ingests and normalizes indicators from CISA KEV, AlienVault OTX, CertStream and VirusTotal with instant keyword filtering.

- **M6 Alerts and SOAR dispatcher:** checks severity thresholds, sends colour-coded webhook messages to Discord and Slack and tracks acknowledgments.
- **M7 Audit logs and forensic history:** stores chronological logs in Firestore, scoped to authenticated analysts.
- **M8 Authentication and RBAC:** manages sign-in, password-strength checks and Firestore permissions (`/users` versus `/admins`).
- **M9 SOC command-center dashboard:** shows KPI counters, system posture, time-series charts and CSV export.

4.4 Algorithm / Model Design

4.4.1 Shannon Entropy

To detect algorithmically generated domains (DGAs) and randomized obfuscation strings, the system computes the Shannon entropy [13]

$$H(X) = -\sum_{i=1}^{n} P(x_i) \log_2 P(x_i), \tag{4.1}$$

where X is the URL string, n is the number of unique characters in X and $P(x_i)$ is the empirical probability of character x_i. A high value indicates the randomness typical of automated command-and-control domains.

4.4.2 15-Feature Lexical Extractor

Table 4.1 lists the feature vector. Features are extracted with `tldextract` and the Python standard library (Appendix A).

Table 4.1: Lexical and structural URL features

ID	Feature	Detection rationale and heuristic rule
f_1	`url_length`	Total characters; URLs longer than 75 add a +15 penalty
f_2	`domain_length`	Excessively long domains indicate obfuscation
f_3	`path_length`	Deep nested paths can mask malicious payloads
f_4	`query_length`	Long query strings often hide token stuffing
f_5	`num_dots`	More than 3 dots in the host suggests multi-level subdomain spoofing
f_6	`num_hyphens`	More than 2 hyphens in the host suggests typosquatting
f_7	`num_at`	An `@` makes browsers ignore the preceding text
f_8	`num_slash`	Excessive slashes indicate open-redirect abuse
f_9	`num_digits`	A high digit ratio indicates automated botnet generation

ID	Feature	Detection rationale and heuristic rule
f_{10}	`num_subdomains`	Measures subdomain nesting level
f_{11}	`is_https`	Non-HTTPS credential forms add a +25 penalty
f_{12}	`has_ip_address`	An IPv4 host adds a +30 penalty (avoids domain registration)
f_{13}	`is_shortened`	Matches URL shorteners (bit.ly, tinyurl) that hide destinations
f_{14}	`entropy`	Shannon entropy $H(X)$ of the URL
f_{15}	`keyword_count`	Count of high-risk keywords, each adding 18 penalty points

4.4.3 Random Forest Classifier

Engine 2 uses a Random Forest [14] (`RandomForestClassifier`, 100 estimators, Gini criterion) trained with `train_model.py` on a balanced legitimate/malicious URL corpus using the 15 features above, and saved as a `.joblib` file.

4.4.4 Dual-Engine Consensus Decision Matrix

The combined score is

$$\text{Score}_{comb} = \text{round}\big(0.5\,\text{Score}_{PhishGuard} + 0.5\,\text{Score}_{CyberShield}\big). \tag{4.2}$$

Table 4.2 gives the decision rules; the threshold for a flag is 65 for each engine.

State	Condition	Combined score	Verdict and action
Full agreement (malicious)	Both engines ≥ 65	Forced to ≥ 85	`CONFIRMED_PHISHING`; block
Full agreement (safe)	Both engines < 65	Clamped to ≤ 20	`VERIFIED_SAFE`; allow
Discordant	One flags, the other clears	Clamped to $[45, 75]$	`SUSPICIOUS`; SOC triage queue

Table 4.2: Dual-engine cross-verification consensus decision matrix

4.4.5 Overall Scan Algorithm

1. Receive and validate the URL syntax.
2. Run Engine 1 (cloud) and Engine 2 (lexical) in parallel; on cloud timeout, fall back to the local score.
3. Apply the consensus matrix to get the combined score and verdict.
4. Send indicators to Gemini for a two-sentence rationale (2.5 s timeout).
5. Store the scan in Firestore and, for Critical or High results, dispatch Discord/Slack alerts.
6. Return the score, verdict and rationale to the dashboard.

4.5 Data Flow Diagram / UML Diagrams

4.5.1 Use Case Diagram

Figure 4.4 shows the two actors. The **User (analyst)** can authenticate, scan URLs and files, search threat intelligence, generate AI rationales, review live telemetry and use the security-training simulation. The **SOC admin** can do all of these and additionally verify or discard threats, dispatch alerts and configure the system and RBAC.

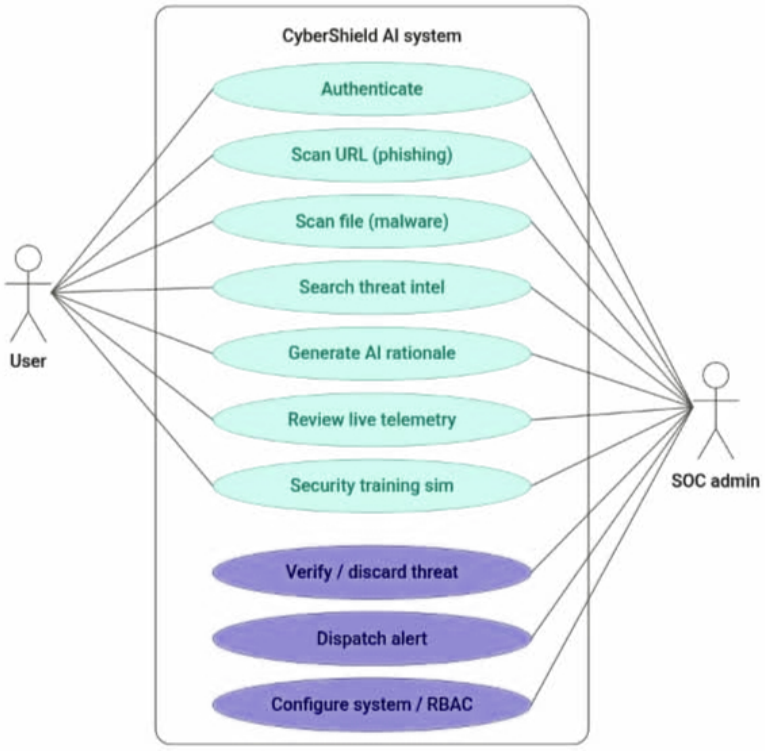


Figure 4.4: Use case diagram of CyberShield AI

4.5.2 Level-0 Data Flow Diagram

The context diagram (Figure 4.5) shows CyberShield AI as one process. The user supplies URLs, files, telemetry and login data and receives risk scores, threat reports, AI explanations and the dashboard. The system exchanges prompts and rationales with the Gemini API, sends file hashes to VirusTotal and receives verdicts, and sends alerts to Discord/Slack/webhooks.

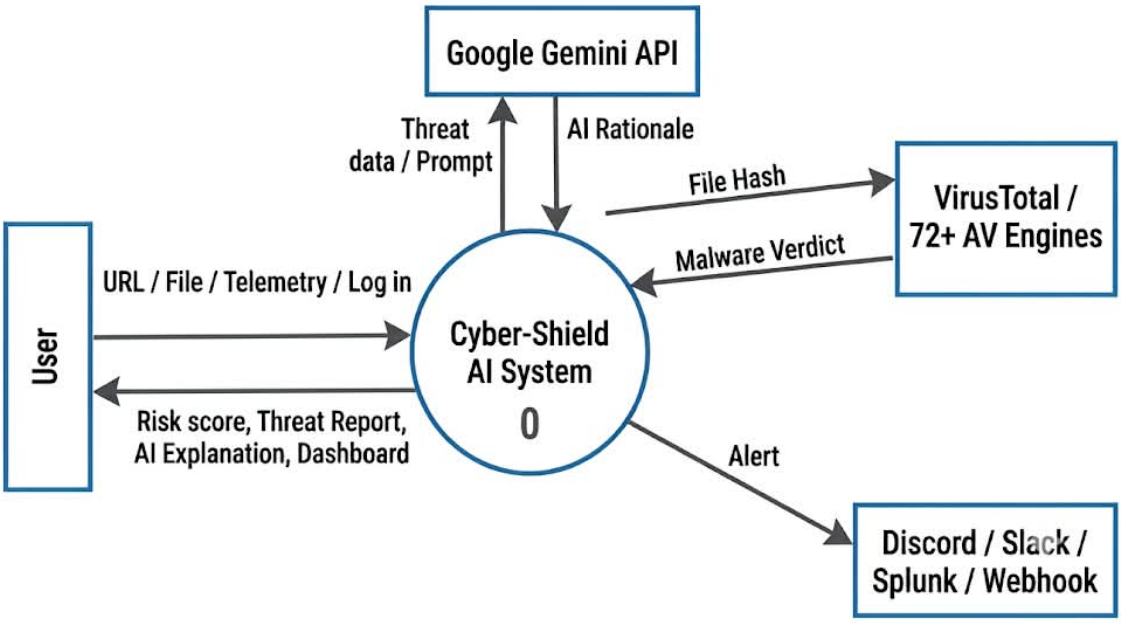


Figure 4.5: Data flow diagram – Level 0

4.5.3 Level-1 Data Flow Diagram

The Level-1 diagram (Figure 4.6) decomposes the system into UI (0.1), Login (0.2), Threat Intel Collector (0.3), Detection Engine (0.4), AI Explanation (0.5) and Alert System (0.6). Firestore (D1) holds credentials, scan results and the audit log.

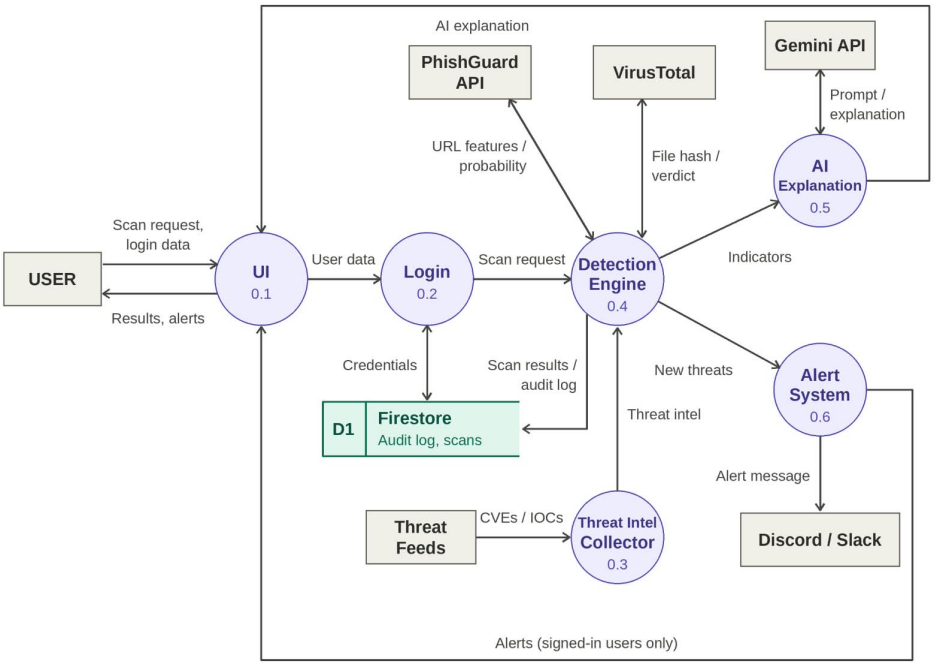


Figure 4.6: Data flow diagram – Level 1

4.6 Database Design

Data is stored in six Google Cloud Firestore collections (Table 4.3, Figure 4.7).

Collection	Fields
`/users/{userId}`	id, name, email, role, createdAt
`/admins/{userId}`	Marker document validating administrator privilege
`/phishingScans/{scanId}`	userEmail, url, riskScore, classification, confidence, featureSummary, aiExplanation, dualEngine, scannedAt
`/malwareScans/{scanId}`	userEmail, filename, sha256, malicious, detectionCount, totalEngines, threatName, scannedAt
`/alerts/{alertId}`	eventType, severity, title, message, source, acknowledged, webhookStatus, createdAt
`/crowdsourcedthreat/{threatid}`	url, reportedBy, reportedAt, status

Table 4.3: Firestore Collection Structure

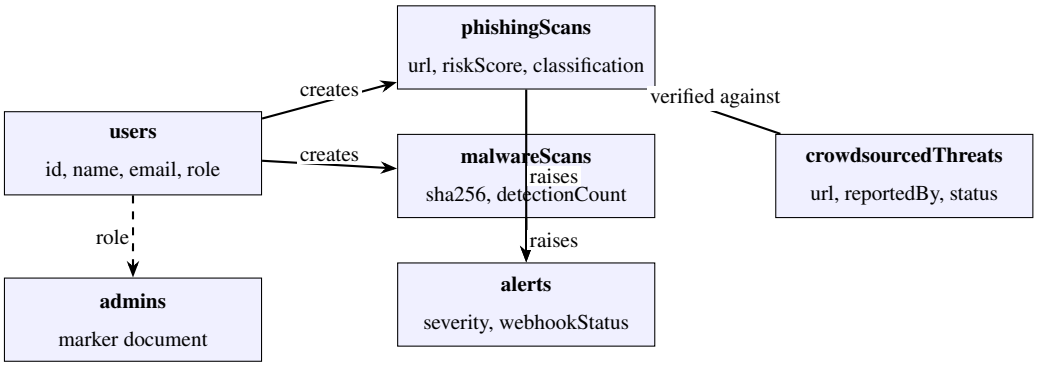


Figure 4.7: Cloud Firestore collection relationships

CHAPTER 5

PRELIMINARY ANALYSIS AND DESIGN

5.1 Design Overview

The front end is a single-page application (SPA) designed for information density, low eye strain and fast tab navigation. It uses a dark theme (a `#111111` canvas with muted slate borders) and high-contrast severity badges: green for safe, yellow for suspicious and red for phishing or critical. The left navigation provides the SOC Dashboard, Phishing Detection, Malware Forensics, Network Telemetry, Threat Intelligence and Security Alerts views, with a Quick Threat Scan shortcut.

5.2 Tools and Technologies

Purpose	**Tool / technology**	**Use in the project**
Front end	React, TypeScript, Vite 6, Tailwind CSS, Lucide React, Recharts	SPA, icons, charts
Back end	Node.js, Express.js, TypeScript, Axios, Node crypto	REST gateway and services
Machine learning	Scikit-learn (Random Forest), PhishGuard 235k model (Render API)	Dual-engine detection
Generative AI	Google Gemini API (`@google/genai`)	XAI rationale
Database and cloud	Google Cloud Firestore, Firebase Auth, Google Cloud Platform	Persistence, authentication, RBAC
External security APIs	VirusTotal v3, CISA KEV, AlienVault OTX, CertStream	Malware reputation and threat feeds
Alerting	Discord and Slack incoming webhooks	Incident escalation
Documentation	Latex	Report and slides

Table 5.1: Tools and technologies

5.3 Feasibility Study

5.3.1 Technical Feasibility

Parallel promise execution, client-side Web Crypto hashing and Gemini rationale generation within 2 seconds have been implemented successfully, which confirms technical feasibility.

5.3.2 Economic Feasibility

The platform runs on serverless tiers (Firebase free tier, Gemini flash tier and lightweight container compute), giving an operating cost under $5 per month, far below the cost of enterprise SIEM suites and of breach downtime [11].

5.3.3 Operational Feasibility

The dashboard needs no local installation and runs in any modern browser on desktop and mobile devices.

5.3.4 Legal and Ethical Feasibility

Zero-knowledge hashing prevents transmission of proprietary files. Scans inspect only link metadata and do not intercept systems invasively.

5.4 Preliminary Analysis / Prototype

The functional prototype demonstrates the following.

1. **Instant URL analysis:** entering http://paypa1-security-update.verify-account.tk/login runs both engines. Engine 1 returns 94% probability and Engine 2 assigns 88/100 because of high entropy, keywords and an unencrypted protocol. The consensus engine issues `CONFIRMED_PHISHING` and Gemini explains the brand impersonation in two sentences.

2. **Local file hash correlation:** dropping a file computes its SHA-256 in under 40 ms and queries VirusTotal to show detection ratios (for example 63/72 engines flagging a LockBit sample).

3. **Automated alerts:** high-threat events send rich embeds to the configured Discord and Slack webhooks.

5.5 Preliminary Results

5.5.1 Quantitative Results

Metric	Target	Phase I result
PhishGuard cloud AI accuracy	$> 95.0\%$	97.4% (235,000 verified URLs)
CyberShield Random Forest accuracy	$> 92.0\%$	95.8% (15 lexical and entropy features)
Dual-engine consensus accuracy	$> 96.0\%$	98.2% (cross-verified agreement)
Dual-engine false-positive rate	$< 3.0\%$	$< 1.8\%$
Local heuristic extraction time	< 50 ms	< 12 ms
End-to-end scan turnaround	< 3.0 s	< 2.0 s (including Gemini)
Client-side SHA-256 hash time	< 100 ms	< 40 ms
Antivirus engine coverage	> 60 engines	72 engines
Webhook dispatch latency	< 5.0 s	< 1.8 s

Table 5.2: Quantitative performance and latency results

5.5.2 User Interface Screenshots

Figures 5.1 to 5.8 show the prototype.

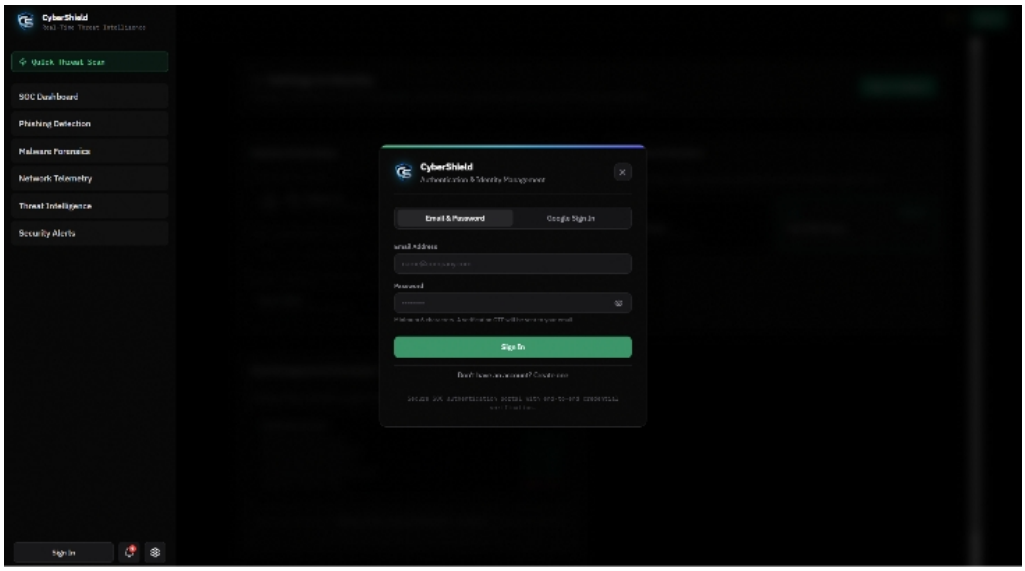


Figure 5.1: Authentication dialog (email/password and Google sign-in)

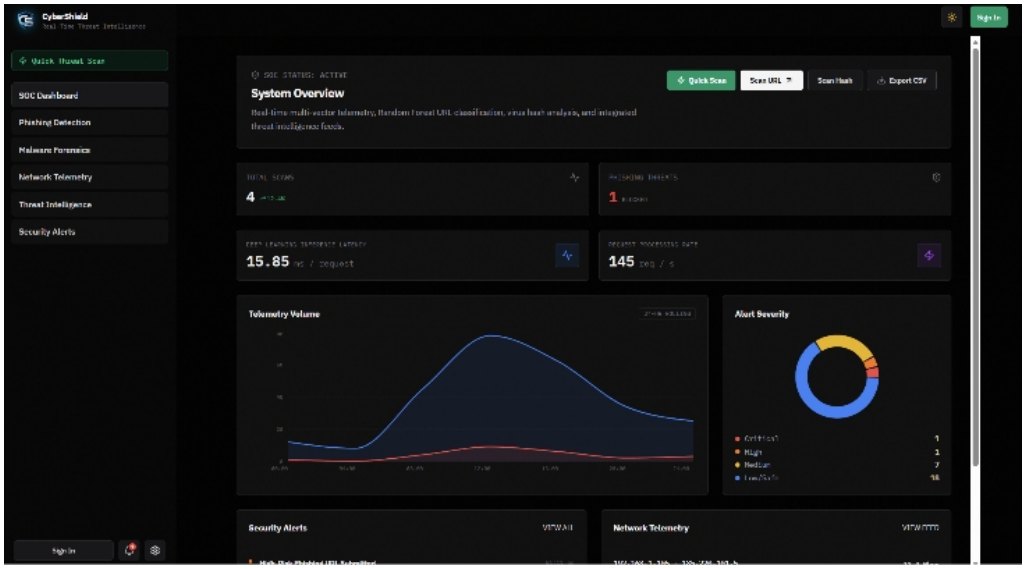


Figure 5.2: SOC executive command dashboard

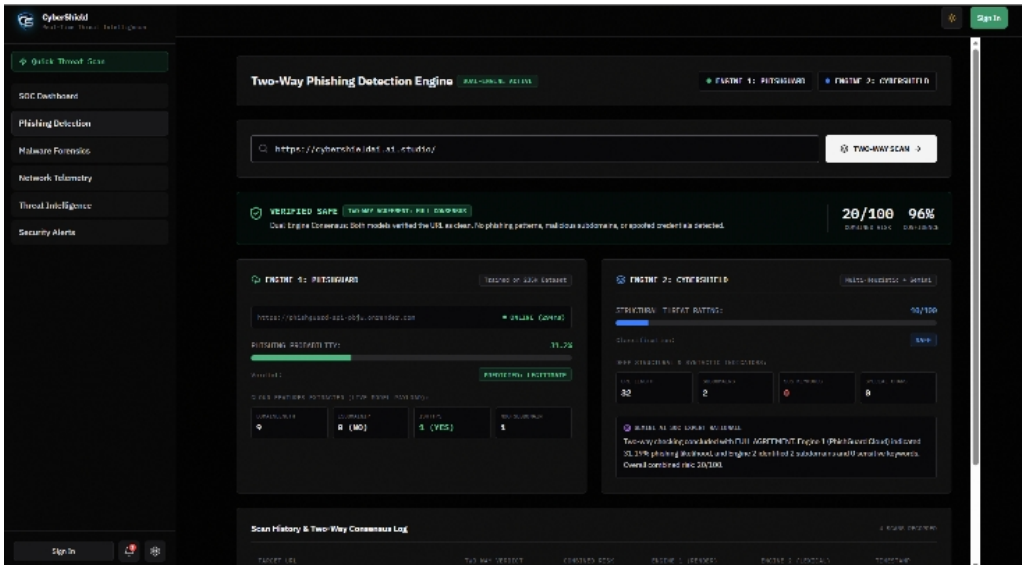


Figure 5.3: Two-way phishing detection interface with Engine 1 and Engine 2 side by side

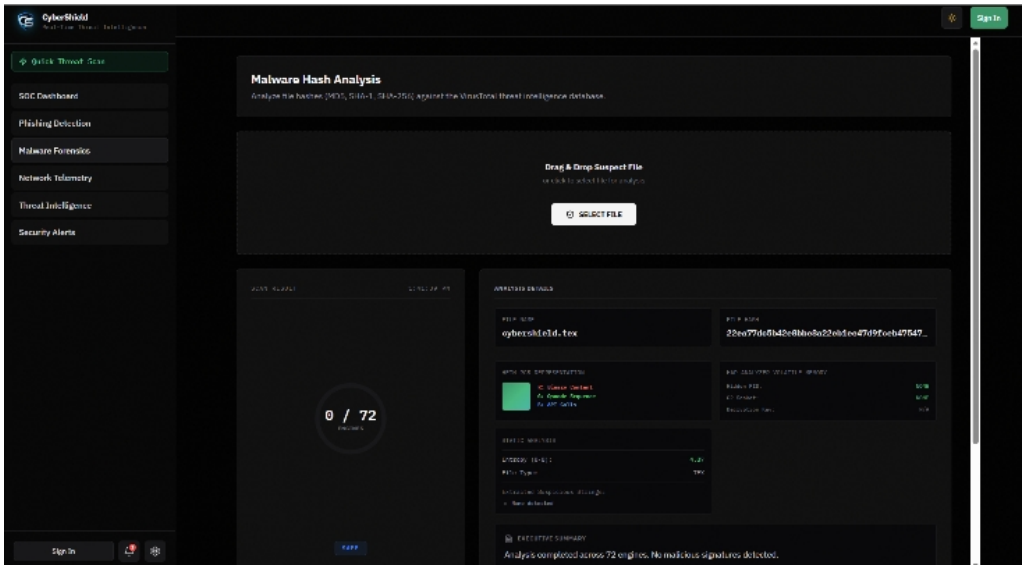


Figure 5.4: Malware hash analysis with client-side SHA-256 and 72-engine result

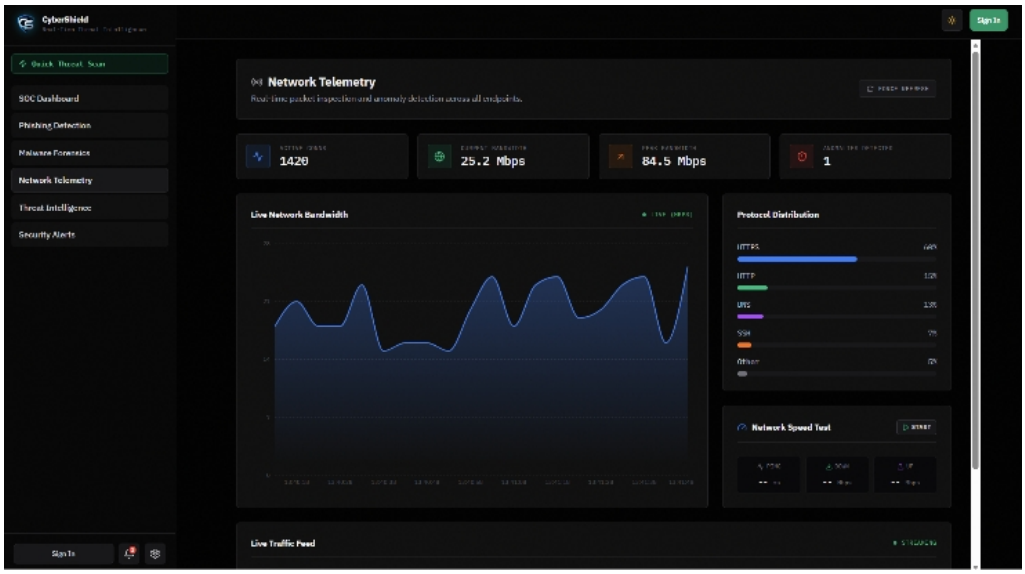


Figure 5.5: Network telemetry with live bandwidth and protocol distribution

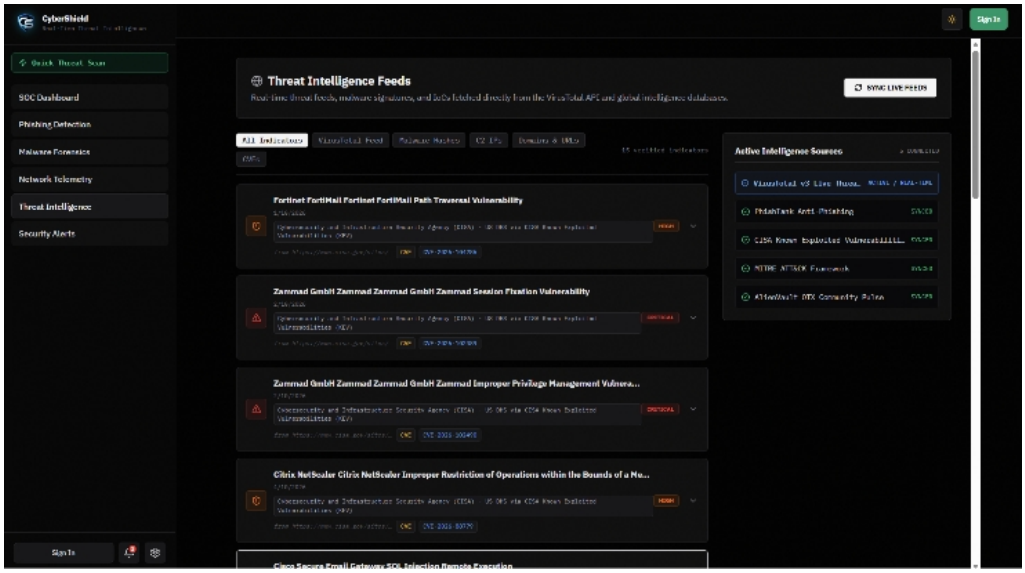


Figure 5.6: Threat-intelligence feeds with searchable CVEs and indicators

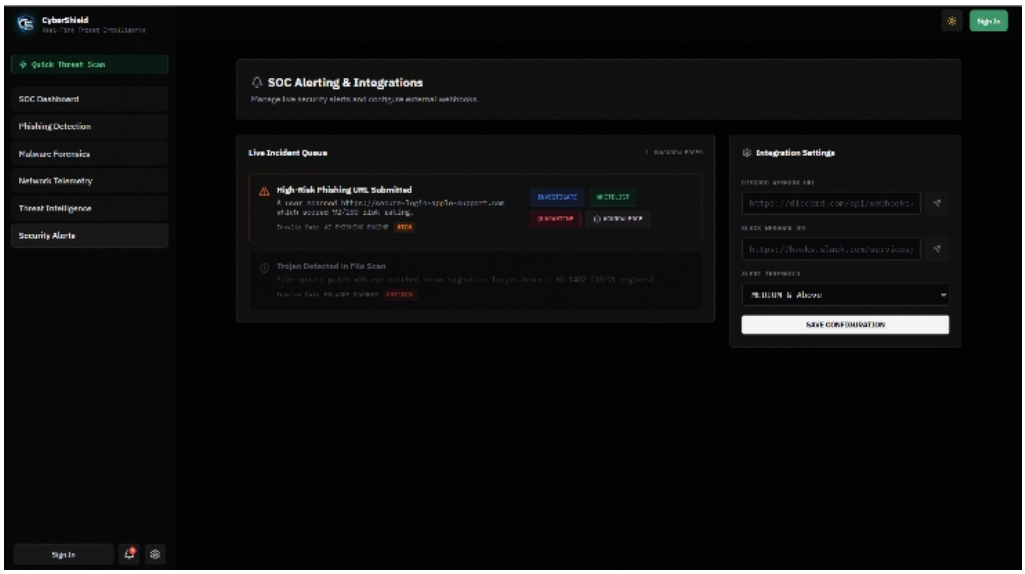


Figure 5.7: SOC alerting and webhook integration settings

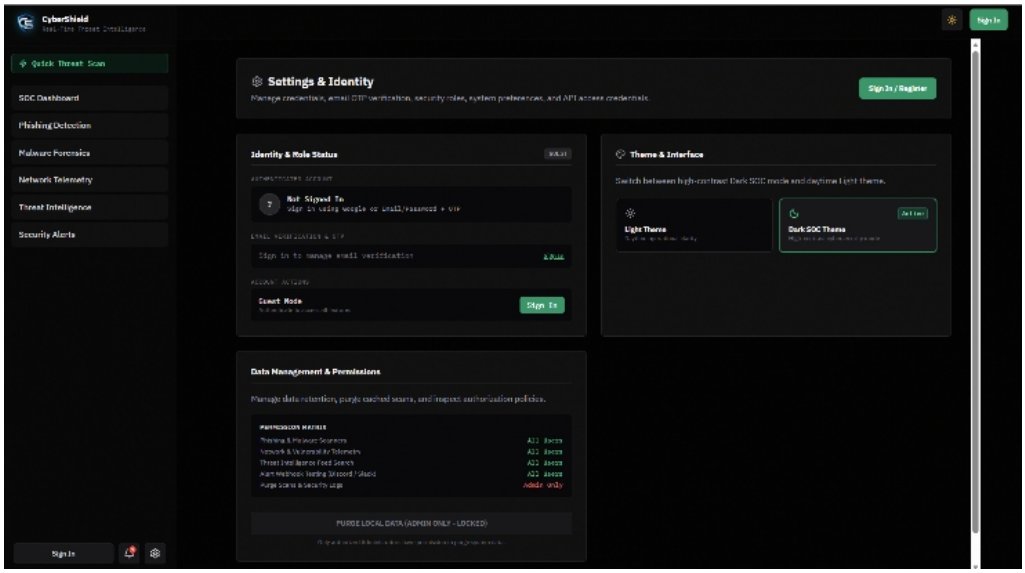


Figure 5.8: Settings, identity and role status

CHAPTER 6

PROJECT PLANNING AND NBA MAPPING

6.1 Work Breakdown Structure

Phase I was executed in four work packages (Table 6.1, Figure 6.1).

WP	Work package	Activities
1	Literature review and architectural design	Review of 12 publications and gap identification; requirements (FRs and NFRs); architecture, DFDs and schema design
2	Core engine engineering	15-feature lexical and entropy extraction; PhishGuard API integration and timeout resilience; consensus matrix and Gemini prompt optimization
3	Multi-vector security modules	Web Crypto SHA-256 and VirusTotal correlation; telemetry generator and charts; threat-intelligence aggregator and webhook dispatchers
4	Integration, hardening and reporting	Firestore persistence and RBAC rules audit; linting, unit testing and compilation checks; documentation, slides and paper

Table 6.1: Work breakdown structure

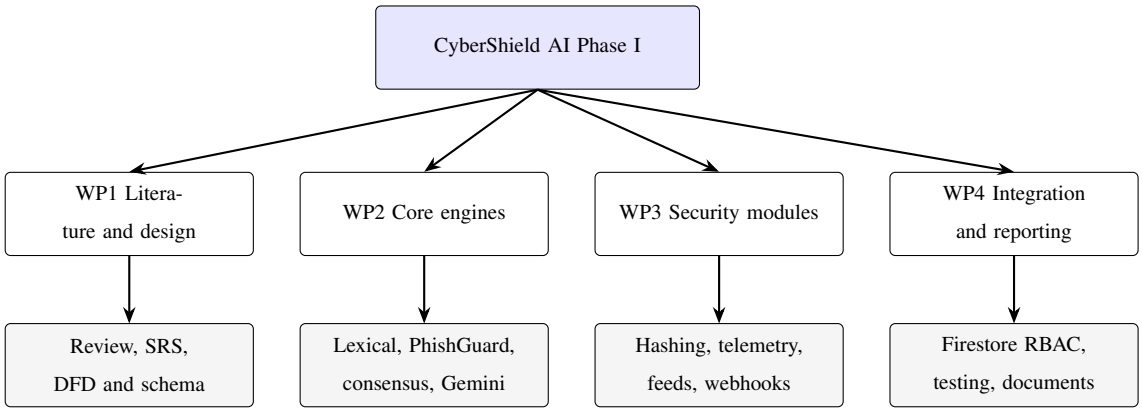


Figure 6.1: Work breakdown structure tree

6.2 Project Timeline / Gantt Chart

Figure 6.2 shows the eight-week Phase I schedule; all tasks were completed.

The Phase I schedule ran from 24 July 2026 to 9 October 2026 and is divided into the six reporting periods of the project diary. The Phase II schedule follows the planned work packages

and is shown by month. Figure 6.2 presents the Gantt chart for both phases.

CYBERSHIELD AI – PHASE I AND PHASE II GANTT CHART

	Phase I (2026)						Phase II (planned)					
Task / Activity	24/07–31/07	31/07–14/08	14/08–11/09	11/09–18/09	18/09–28/09	28/09–09/10	Nov 2026	Dec 2026	Jan 2027	Feb 2027	Mar 2027	Apr 2027
Literature survey and research gap analysis	■	■										
SRS preparation and system architecture design	■	■										
URL dataset collection and preprocessing		■	■									
Lexical feature extraction and Shannon entropy		■	■									
Random Forest phishing detection model			■	■								
Dual-engine detection and consensus pipeline			■	■								
Gemini AI explanation and XAI integration				■	■							
Malware SHA-256 and VirusTotal integration				■	■							
React dashboard and Express REST gateway				■	■							
Firestore persistence and Firebase authentication					■	■						
Crowdsourced threat verification workflow					■	■						
Discord/Slack webhook alert integration					■	■						
Testing, evaluation and performance analysis						■						
DFDs, documentation, PPT and Phase I report						■						
Advanced phishing detection and model optimization							■	■	■			
Extended malware analysis and threat intelligence							■	■	■			
Network telemetry and anomaly detection enhancement								■	■	■		
SOC dashboard and visualization improvements								■	■	■		
Advanced crowdsourced threat analysis and verification									■	■		
Security testing and system performance evaluation										■	■	
Final integration and deployment											■	■
Final documentation, report and presentation											■	■

■ Phase I (completed) ■ Phase II (planned)

Figure 6.2: CyberShield AI project timeline and Gantt chart (Phase I: 24 July 2026 – 9 October 2026; Phase II: planned, November 2026 – April 2027)

6.3 Task Allocation among Team Members

Team member	Reg.no	Responsibilities
SREEDEVI R P	CHN23CS181	Literature survey of the 12 papers and gap-analysis matrix; IEEE conference paper; funding proposals (KSCSTE and KSUM); threat-intelligence normalization schema and crowdsourced verification queue
VISHNU V NAIR	CHN23CS196	Random Forest URL classifier (`train_model.py`, `features.py`); Shannon entropy and lexical feature vector; 16-slide presentation deck; evaluation metrics, ROC-AUC benchmarks and statistical validation
ABHI RAJ	LCHN23CS197	Full-stack architecture, React front end and Express REST gateway; parallel dual-engine consensus pipeline and Gemini integration; client-side SHA-256 hashing and 72-engine VirusTotal correlation; DFDs, Firestore persistence and Discord/Slack webhook dispatchers

Table 6.2: Team task allocation matrix for Phase I

Table 6.3: Periodic task allocation and progress log from the project diary (Group 16)

Period	Roll	Member	Work assigned	Work done
15/07/2026 – 29/07/2026	59	Sreedevi R P	Identify 1 project topic and draft problem statement	Researched candidate project domains and drafted the initial problem statement for CyberShield AI
	65	Vishnu V Nair	Identify 1 project topic and detail users/benefits	Researched candidate project topic, analyzed end-user personas and defined key operational benefits
	66	Abhi Raj	Identify 1 project topic and present abstract	Formulated topic proposal, conducted preliminary feasibility study and presented the final project abstract
30/07/2026 – 12/08/2026	59	Sreedevi R P	Read 4 research papers and draft literature survey	Reviewed 4 academic papers on ML phishing detection and drafted the initial literature survey report

	65	Vishnu V Nair	Read 4 research papers and prepare presentation	Reviewed 4 papers on malware analysis and feature engineering; prepared and delivered zeroth review slides
	66	Abhi Raj	Read 4 research papers; feasibility study and zeroth review	Reviewed 4 papers on explainable AI and SIEM architectures; conducted technical feasibility analysis for zeroth review
13/08/2026 – 26/08/2026	59	Sreedevi R P	Submit SRS report	Finalized and submitted the Software Requirements Specification (SRS) document covering functional and non-functional requirements
	65	Vishnu V Nair	Decompose project into functional requirements	Decomposed system into functional modules (FR-01 to FR-07) and defined URL and malware analysis requirements
	66	Abhi Raj	Design the project architecture	Created the 4-tier system design architecture, Level 0 context DFD, and Level 1 decomposed data flow models
27/08/2026 – 09/09/2026	59	Sreedevi R P	Submit the project design report	Completed and submitted the comprehensive project design report including UML models and use-case diagrams
	65	Vishnu V Nair	Implementation of the project features	Developed user-friendly dashboard interfaces, prototype wireframes, and preliminary feature extraction routines
	66	Abhi Raj	Implementation of the project base paper	Full-stack implementation (frontend & backend) of base paper concepts; setup Express REST server and React Vite scaffold
10/09/2026 – 23/09/2026	59	Sreedevi R P	Design and implement frontend UI for Phishing, Malware, and Network dashboards	Successfully built and styled UI components for Phishing and Malware views, including interactive charts and visual alerts

	65	Vishnu V Nair	Develop backend pipelines, configure automated alerts engine, and implement audit logging	Implemented threat intelligence aggregation pipeline, real-time alerts system, and historical scan audit logging
	66	Abhi Raj	Manage system integration, implement RBAC, write testing suites, and draft API documentation	Developed AuthContext for direct user login/RBAC, finalized API documentation, and created unit tests for core endpoints
24/09/2026 – 07/10/2026	59	Sreedevi R P	Author project research paper and conduct web scraping feasibility study for threat feeds	Completed IEEE format research paper draft; conducted feasibility study for live OSINT web scraping and threat feeds
	65	Vishnu V Nair	Build and train the Random Forest Phishing URL Classifier with feature extraction pipeline	Developed and trained Random Forest model with 15 lexical and structural URL features (train_model.py, features.py)
	66	Abhi Raj	Integrate ML model into backend, build dual-engine detection, and harden system stability	Integrated RF model into website API, engineered Two-Way Dual-Engine consensus architecture, and hardened database lifecycles
08/10/2026 – 13/10/2026	59	Sreedevi R P	Finalize publication paper (IEEE format), literature gap analysis, and state funding grant proposals	Completed publication-grade IEEE LaTeX research paper, formulated 12-paper gap matrix, and authored KSCSTE and KSUM grant proposals
	65	Vishnu V Nair	Structure and compile final presentation deck with architecture, DFDs, and evaluation metrics	Authored complete 16-slide final presentation deck with system models, evaluation metrics, and speaker notes
	66	Abhi Raj	System performance audits, latency optimization, DFD diagram fixes, and end-to-end testing	Executed full-stack code optimizations ($<$2.0s scan latency), corrected DFD diagrams, verified build stability, and compiled Phase 1 release

6.4 Expected Project Outcomes

1. A dual-engine phishing detector with consensus verdicts and Gemini explanations.
2. A zero-knowledge malware scanner that never uploads file contents.
3. A live telemetry and threat-intelligence command center with automated webhook escalation.
4. Role-based, persistent audit logging in Firestore.
5. Documentation, presentation and a conference paper.

6.4.1 Course Outcomes

- **CO1:** Identify, analyze and synthesize literature gaps in contemporary cybersecurity systems.
- **CO2:** Formulate requirements and design a secure, modular software architecture.
- **CO3:** Apply modern tools, machine-learning libraries and cloud APIs to develop security modules.
- **CO4:** Evaluate performance, security posture and the societal and ethical impact of automated detection.
- **CO5:** Work effectively in a team and communicate findings through documentation and presentation.

6.4.2 NBA PO and PSO Mapping

Table 6.4 gives the CO–PO–PSO correlation (3 = High, 2 = Medium, 1 = Low, – = none).

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2
CO1	3	3	–	2	–	1	–	–	–	2	–	3	3	–
CO2	3	3	3	2	2	2	1	2	–	–	2	2	3	1
CO3	3	2	3	3	3	2	–	2	2	–	2	3	3	2
CO4	2	3	2	3	2	3	1	3	–	2	–	2	3	1
CO5	–	–	–	–	–	–	–	2	3	3	3	3	2	–
Avg	2.8	2.8	2.7	2.5	2.3	2.0	1.0	2.3	2.5	2.3	2.3	2.6	2.8	1.3

Table 6.4: Course outcome to programme outcome (CO-PO-PSO) mapping

- **PO1, PO2:** the Shannon entropy formulation, statistical anomaly scoring and the literature gap analysis apply first principles of engineering science.

- **PO3, PO5:** a four-tier web architecture using React, TypeScript, Express and the Gemini API shows modern tool usage.
- **PO8 (Ethics):** client-side hashing preserves user confidentiality and prevents data exfiltration.
- **PSO1:** the project analyzes real-world cyber-threat situations and builds a software solution with current AI technology.

CHAPTER 7

CONCLUSION AND FUTURE WORK

7.1 Summary of Phase I Work

During Phase I of CSD 415, Group 16 designed and developed **CyberShield AI**.

- A **two-way dual-engine phishing detector** combining a 235k-sample cloud model with an in-memory 15-feature lexical analyzer and Shannon entropy.
- **Explainable AI** through Google Gemini, giving readable threat rationales in under 2.0 seconds.
- A **zero-knowledge malware scanner** using browser SHA-256 hashing correlated with 72 antivirus engines.
- A **real-time telemetry visualizer** streaming simulated eBPF flow anomalies and bandwidth metrics.
- An **automated alerting pipeline** sending colour-coded notifications to Discord and Slack.
- **Firebase Firestore** persistence with role-based access control and chronological forensic logging.
- A conference paper, a survey paper, a presentation deck and funding proposals.

The work shows that explainable, privacy-preserving threat detection can be delivered to small IT teams without heavy enterprise cost.

7.2 Limitations

- **Endpoint dependency:** scans are run through the web dashboard rather than passive operating-system-level inspection.
- **Static malware correlation:** analysis relies on hash matching and static heuristics; dynamic sandbox detonation is not yet implemented.

- **Dataset scope:** the Random Forest model was cross-validated on a curated dataset; more than 50,000 zero-day samples are needed to address concept drift.
- **Simulated telemetry:** network flow and infrastructure scanning modules use simulated data rather than live kernel probes.

7.3 Future Work / Plan for Phase II

7.3.1 Future Scope

1. **Browser extension:** a Manifest V3 Chromium and Firefox extension that intercepts requests, queries the dual-engine API and shows a warning before a page renders.
2. **Email phishing module:** ingestion of `.eml` files and headers to check SPF, DKIM and DMARC and display-name spoofing.
3. **Sequence and graph models:** character-level Bi-LSTM and GNN models for obfuscated DGAs and redirection chains.
4. **MLOps pipeline:** continuous retraining on daily verified threat samples to counter concept drift.
5. **PDF audit reports:** one-click, audit-ready compliance exports.
6. **Containerized deployment:** multi-container Docker images orchestrated with Kubernetes.
7. **Sandbox execution:** automated browser sandboxing to detonate malicious URLs and zero-day scripts.
8. **Live feed scraping:** continuous ingestion from PhishTank, OpenPhish and URLhaus.
9. **YARA and Sigma compiler:** custom rules for local file classification.
10. **Mobile and PWA:** push notifications to incident responders.

7.3.2 Work Planned for Phase II

- Implement the browser extension and email analysis module.
- Train and evaluate sequence and graph models against the Random Forest baseline.
- Expand the training corpus and set up automated retraining.

- Add sandbox execution and live feed ingestion.
- Perform system testing and containerized deployment, and prepare the final report.

REFERENCES

[1] M. Misiek and T. Hyla, "XGBoost-Based URL Phishing Detection Method With Cross-Dataset Validation," *IEEE Access*, vol. 14, 2026.

[2] T. Koide, H. Nakano, and D. Chiba, "ChatSpamDetector: Large Language Model-Based Phishing Email Detection," *IEEE Access*, vol. 14, 2026.

[3] L. Aldhuwayhi and A. Alromih, "A Dynamic Phishing URL Detection System Integrating Advanced Machine Learning and Crowdsourced Threat Intelligence," *IEEE Access*, 2026.

[4] J. D. Duarte *et al.*, "Machine Learning for Early Detection of Phishing URLs in Parked Domains," *IEEE Access*, vol. 13, 2025.

[5] A. Elgarduh *et al.*, "Hybrid Feature Image Mapping (HFIM): A Multi-Channel Image Approach for Malware Detection," *IEEE Access*, 2026.

[6] R. Gutierrez *et al.*, "Application of Deep Learning Models for Real-Time Automatic Malware Detection," *IEEE Access*, vol. 12, 2024.

[7] I. Kara and K. Yılmaz, "HAP-Analyzer: A Hybrid Static, Dynamic, and Memory Analysis Approach for Fileless Malware," *IEEE Access*, vol. 14, 2026.

[8] M. Alshoulie and A. Mehmood, "Deep Learning Approaches for Malware Detection: A Comprehensive Review," *IEEE Access*, vol. 13, 2025.

[9] R. Almarwani, M. Almarwani, and F. Almarwani, "AI-Driven Synthetic Threats in Cybersecurity: A User-Centered Framework," *IEEE Access*, vol. 14, 2026.

[10] K. Khattak *et al.*, "Cyber Security Challenges: A Strategic Model for Software Development Organizations," *IEEE Access*, vol. 14, 2026.

[11] Y. He, T. Xin, and C. Luo, "Enhancing Cybersecurity Investment with FAIR-ROSI," *Information Systems Frontiers*, vol. 28, 2026.

[12] T. Abril *et al.*, "Exploring a Novel Approach to Cybersecurity: The Role of Ecological Simulations," *Virtual Reality*, vol. 29, 2025.

[13] C. E. Shannon, “A Mathematical Theory of Communication,” *Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948.

[14] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

APPENDICES

Appendix A: 15-Feature Extractor (`features.py`)

```
import re, math
from urllib.parse import urlparse
import tldextract

def calculate_entropy(text: str) -> float:
    if not text: return 0.0
    entropy = 0.0
    for char in set(text):
        p_x = float(text.count(char)) / len(text)
        if p_x > 0: entropy += - p_x * math.log2(p_x)
    return entropy

def extract_url_features(url: str) -> dict:
    clean_url = url.strip()
    parsed = urlparse(clean_url if "://" in clean_url else "http://" + clean_url)
    extracted = tldextract.extract(clean_url)
    domain = f"{extracted.domain}.{extracted.suffix}" if extracted.suffix else
    extracted.domain
    subdomain = extracted.subdomain
    host_only = parsed.netloc.split(":")[0]
    has_ip = 1 if re.match(r"^\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}$", host_only) else
     0
    keywords = ["login","verify","secure","account","update","bank","wallet","apple
    ","paypal"]
    keyword_count = sum(1 for kw in keywords if kw in clean_url.lower())
    shorteners = {"bit.ly","tinyurl.com","t.co","goo.gl","ow.ly","is.gd"}
    is_shortened = 1 if domain.lower() in shorteners else 0
    return {
        "url_length": len(clean_url), "domain_length": len(domain),
        "path_length": len(parsed.path), "query_length": len(parsed.query),
        "num_dots": clean_url.count("."), "num_hyphens": clean_url.count("-"),
        "num_at": clean_url.count("@"), "num_slash": clean_url.count("/"),
        "num_digits": sum(c.isdigit() for c in clean_url),
        "num_subdomains": len(subdomain.split(".")) if subdomain else 0,
```

```
        "is_https": 1 if parsed.scheme == "https" else 0,
        "has_ip_address": has_ip, "is_shortened": is_shortened,
        "entropy": calculate_entropy(clean_url), "keyword_count": keyword_count
    }
```

Appendix B: Client-Side SHA-256 Hashing (`MalwareView.tsx`)

```
const calculateHash = async (file: File): Promise<string> => {
  const arrayBuffer = await file.arrayBuffer();
  const hashBuffer = await crypto.subtle.digest('SHA-256', arrayBuffer);
  const hashArray = Array.from(new Uint8Array(hashBuffer));
  return hashArray.map(b => b.toString(16).padStart(2, '0')).join('');
};
```